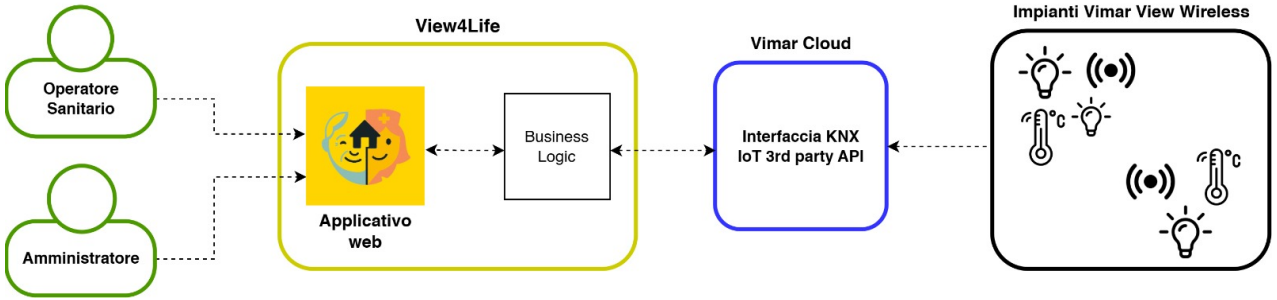
Presentazione architettura

Gruppo 1 - A.A 2025/2026

20/04/2026

Obiettivo capitolato

- Capitolato C9 (Vimar View4Life): Piattaforma per la gestione degli impianti Smart nelle residenze protette;
- Sistema di gestione allarmi e utenti (OSS e Amministrazione) che possono interagire con gli allarmi (gestione e risoluzione)
- Visualizzazione di statistiche sui consumi, con relativi consigli per ridurli, e sugli allarmi.



Architettura di deployment

- Frontend: *Single Page Application* in Angular, servita da *nginx*;
- Backend: monolite realizzato con NestJS.

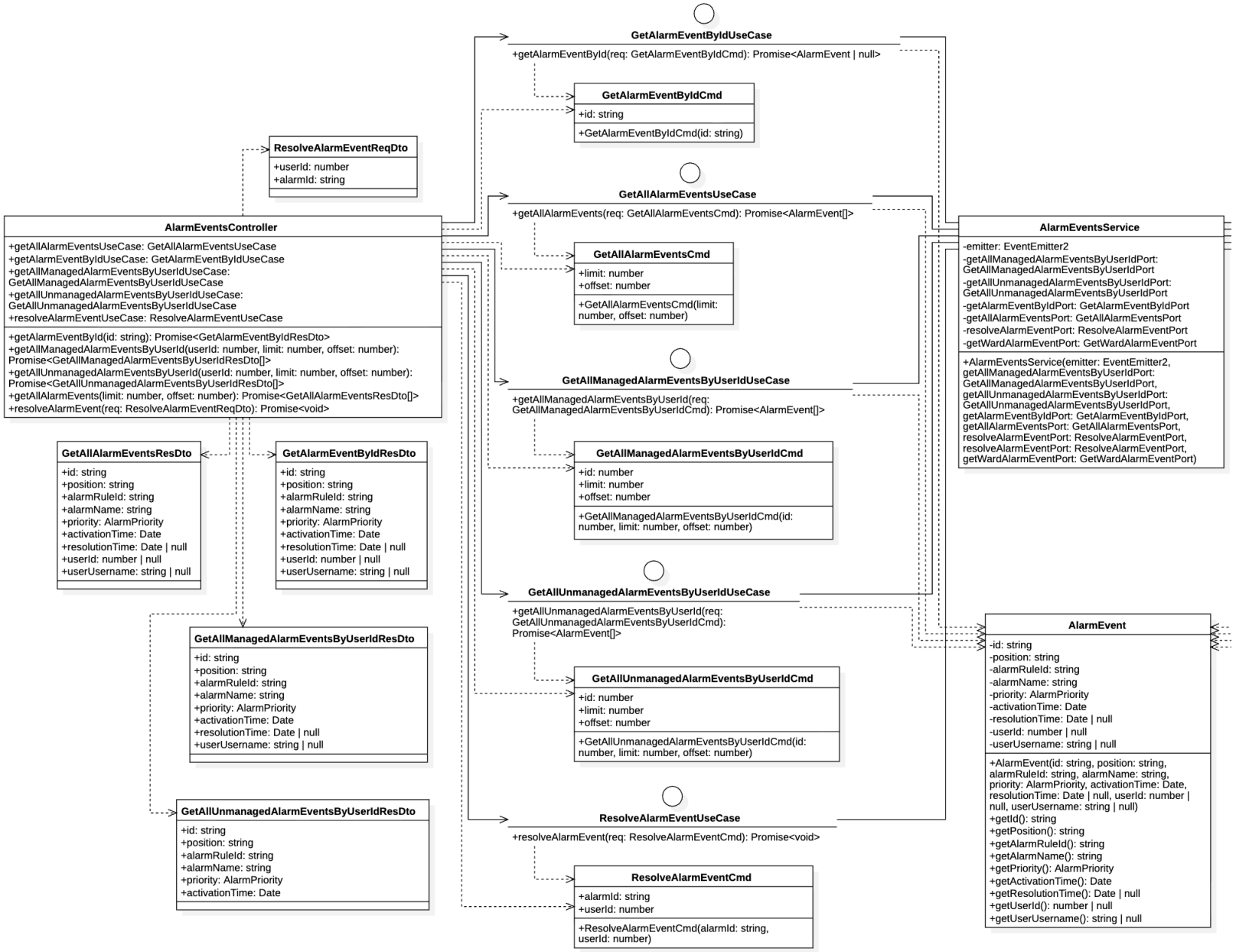
Entrambi, come il database *PostegreSQL*, sono containerizzati e orchestrati da *Docker Compose*.

Attualmente tutti e tre i container sono in esecuzione in un singolo nodo.

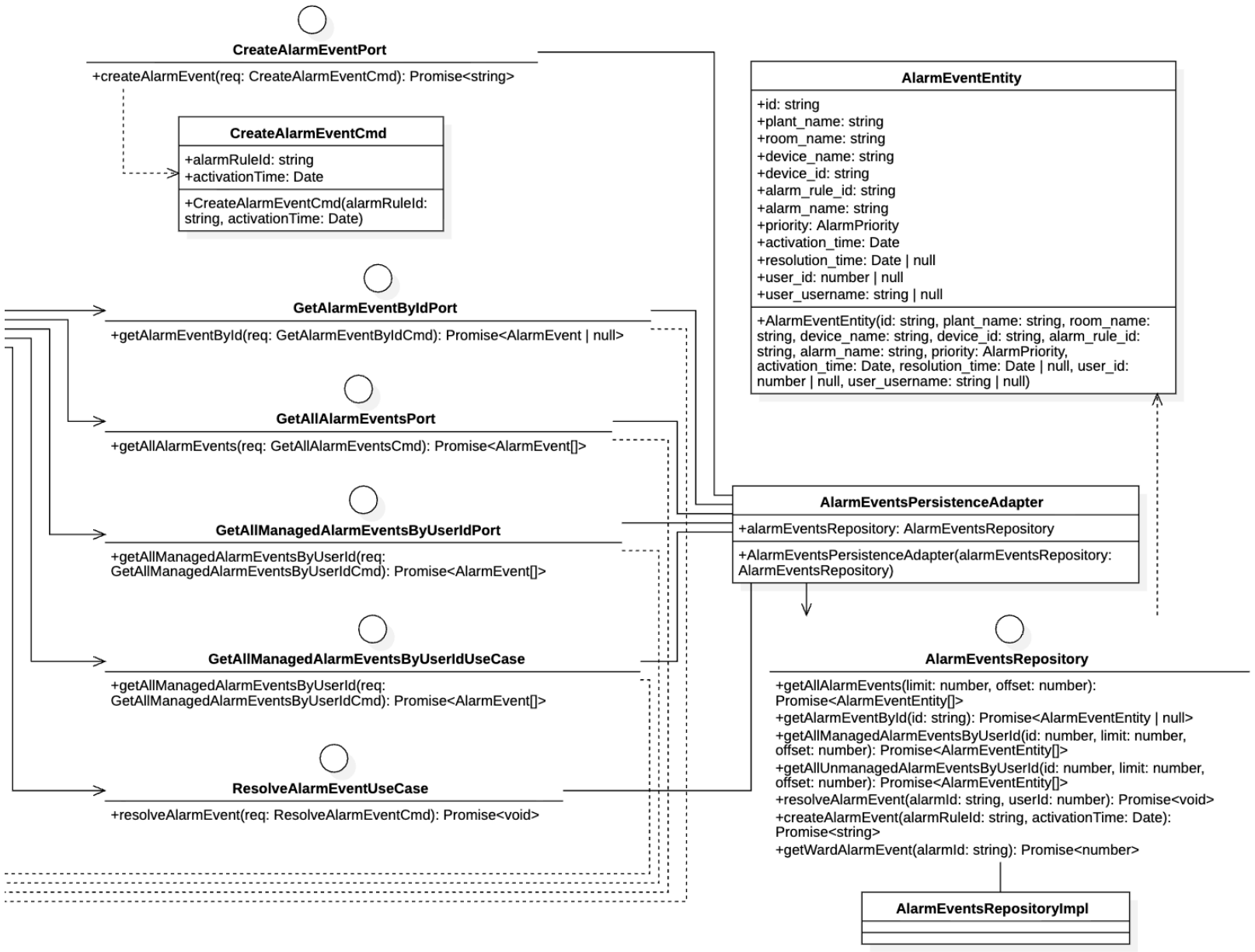
Backend: Architettura esagonale

- Problema:
  - La logica di business del *backend* dipende dai dettagli infrastrutturali (*Persistent Logic*);
  - Difficile testare il *backend* senza dover istanziare l'intera infrastruttura.
- Soluzione: **Architettura Esagonale**
  - Il dominio non dipende da componenti esterne;
  - Le **Ports** sono interfacce *TypeScript* che il dominio espone, gli **Adapters** le implementano;
  - Le dipendenze tra i moduli sono risolte tramite il *pattern* **Dependency Injection**.

UML Alarm (Backend) (1/2)



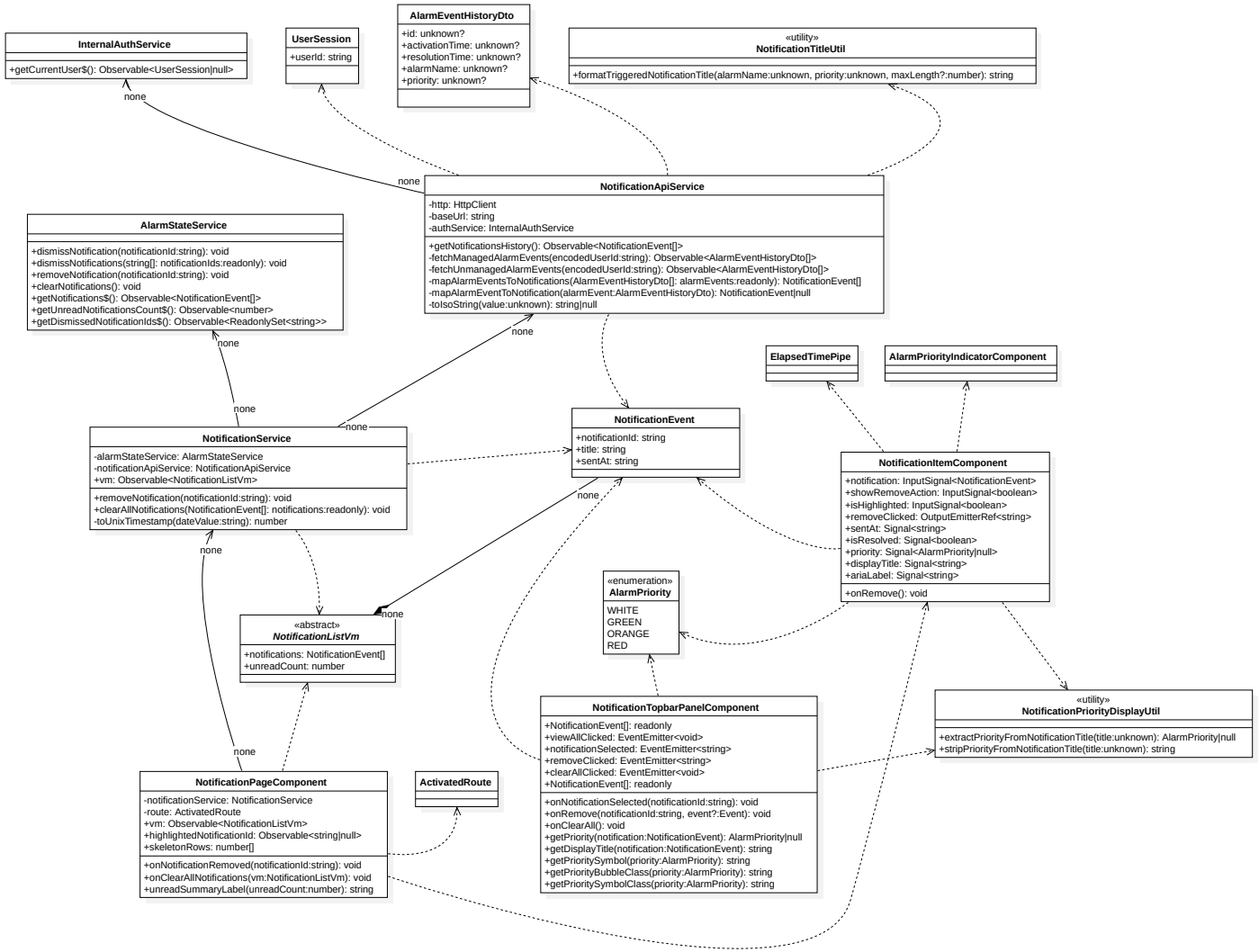
UML Alarm (Backend) (2/2)



Frontend: Architettura Model-View-ViewModel

- Problema:
  - Logica di presentazione legata all'interfaccia utente;
  - Ogni cambiamento della vista rischia di impattare la logica di business.
- Soluzione: **Architettura MVVM** che separa le tre responsabilità:
  - **Model**: i dati e la logica di dominio;
  - **View**: il template di Angular;
  - **ViewModel**: un servizio che espone stato e comandi alla View tramite **Observable**.

UML Notification (Frontend)



Pattern - Dependency Injection (1/2)

- Problema:
  - I moduli sono fortemente accoppiati tra di loro;
  - Il codice risulta difficile da testare e da manutenere.
- Soluzione: pattern **Dependency Injection**
  - Le dipendenze vengono fornite tramite un *injector*;
  - In *TypeScript (Nest JS)* il pattern è supportato nativamente tramite il decoratore *@Injectable()* e il sistema *providers*.

Pattern - Dependency Injection (2/2)

```
providers: [
    { provide: CREATE_ALARM_RULE_USE_CASE, useClass: AlarmRuleService },
    { provide: CREATE_ALARM_RULE_PORT, useClass: AlarmRulesPersistenceAdapter },
    { provide: DELETE_ALARM_RULE_USE_CASE, useClass: AlarmRuleService },
    { provide: DELETE_ALARM_RULE_PORT, useClass: AlarmRulesPersistenceAdapter },
    { provide: GET_ALL_ALARM_RULES_USE_CASE, useClass: AlarmRuleService },
```

```
@Injectable()
export class AlarmRuleService
  implements
    CreateAlarmRuleUseCase,
    GetAlarmRuleByIdUseCase,
    GetAllAlarmRulesUseCase,
    UpdateAlarmRuleUseCase,
    DeleteAlarmRuleUseCase,
    CheckAlarmRuleUseCase
{
```

Pattern - Observer (1/3)

- Problema:
  - I componenti *Angular* devono reagire a eventi asincroni (*WebSocket*) e sincronizzare lo stato *UI*.
- Soluzione: pattern **Observer**
  - Implementato tramite *RxJS*: le componenti si iscrivono a *Observable* senza conoscere la sorgente dei dati;
  - Gli *Observable* gestiscono in modo uniforme eventi asincroni e aggiornamenti di stato *UI*;
  - La comunicazione tra componenti avviene tramite *Subject* e *BehaviorSubject* eliminando ogni accoppiamento diretto.

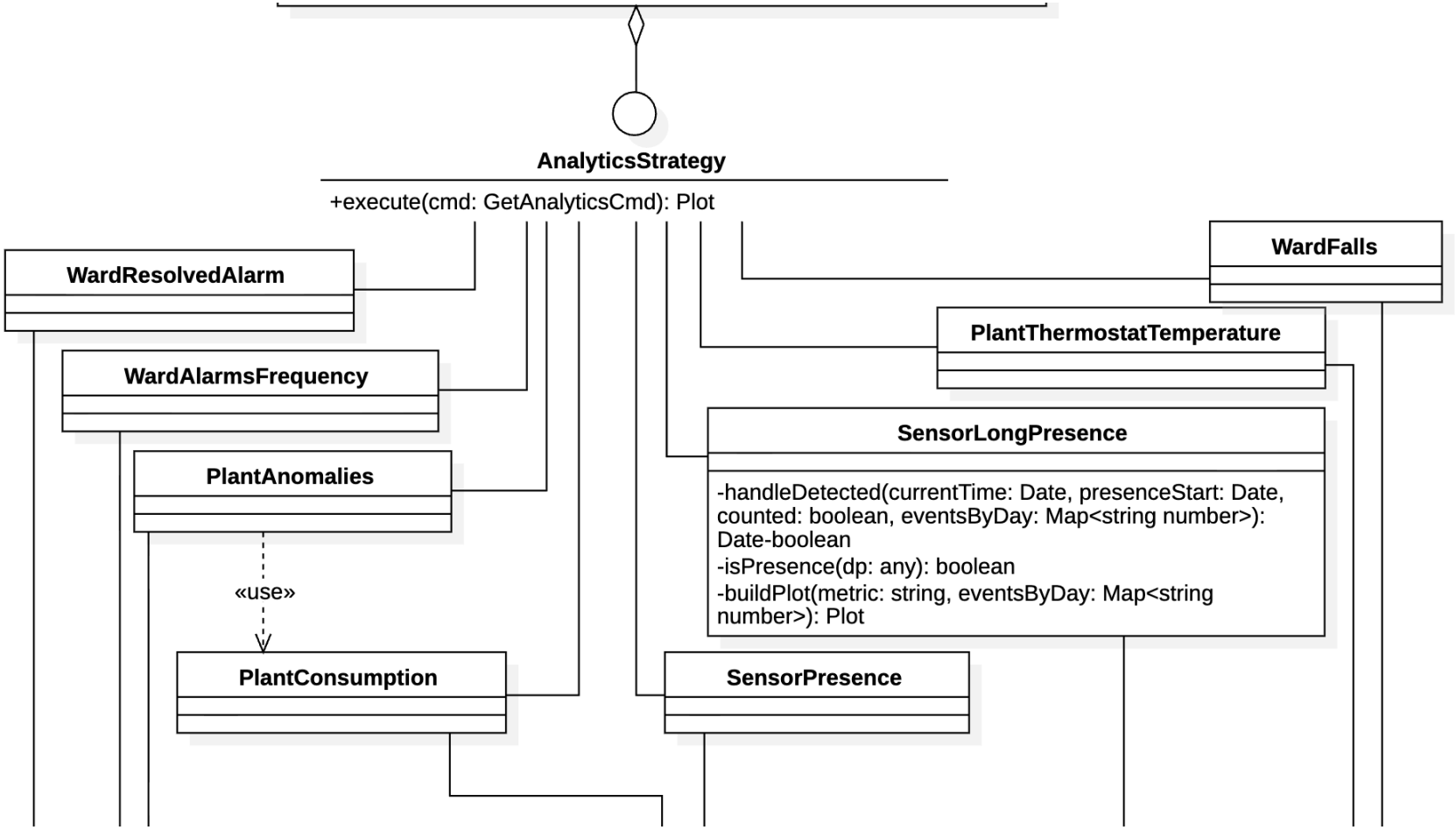
Pattern - Observer (2/3)

UML

Pattern - Observer (2/3)

CODICE

Pattern - Strategy (1/2)

- Problema:
  - Visualizzare un'insieme di *analytics* relative agli impianti;
  - Ogni *analytics* ha un grafico specifico, basato su dati differenti (Es. consumo della luce, temperatura media del termostato, ecc.);
  - Deve essere possibile modificare le *analytics* se necessario.
- Soluzione: **Pattern Strategy**
  - Permette di cambiare a *runtime* l'algoritmo per il calcolo delle *analytics*;
  - È stata realizza un'interfaccia `AnalyticsStrategy` che fornisce il metodo `execute`, che le *strategies* implementano.

Pattern - Strategy (2/2)